

*Aarhus Universitet
ETTML projekt*

Smart Bike Light



Rasmus Gramstrup 202206674

Underviser: Morten Opprud Jakobsen

Dato: 26. maj 2025

Indholdsfortegnelse

1	Forord	1
2	Indledning	2
3	Projekt beskrivelse	3
4	Krav analyse	4
5	System design	5
6	Implementation	6
6.1	Data indsamling	6
6.2	Data nedsampling	10
6.3	Edge Impulse	11
6.4	Data behandling	12
6.5	ML model analyse samt træning	14
7	Test & Verification	20
7.1	Emlearn	20
7.2	C kode	21
8	Konklusion	24
10	Litteratur	25



1 Forord

Denne projektrapport er formuleret af Rasmus Gramstrup, studerende på Aarhus Universitet, der læser til Diplomingeniør i Elektronik. Denne rapport beretter og påviser den studerendes arbejde og udførelse af ETTML projekt på 6. semester.

Projektrapporten grunder i angivne krav for ETTML projekt, fra Aarhus Universitet. Projektrapporten afleveres senest d. 26-05-2025, efterfulgt af det mundtlige forsvar d. 03-06-2025.



2 Indledning

Aarhus er en stor cykel by, på grund af mange forskellige parametre. Både mangel på offentligt transport og prisen, men også manglende parkerings muligheder for biler og de stigende priser for det. Derfor er det oplagt at cykle i Aarhus. Kommunen har også været gode til at anlægge gode cykelstier som sikre plads og sikkerhed for cyklisterne. Men et punkt hvor der mangler sikkerhed, er visningen af håndtegn fra cyklisterne. Det er både visning af drejning til henholdsvis høje og venstre, men også især bremsning. Dette giver problemer og frustrationer for bilister som ikke kan se om cyklisten stopper ved et lyskryds, men også andre cyklister som får svinget ud foran hinanden.

Der kan være mange forskellige årsager til hvorfor cyklister ikke får givet tegn i trafikken. Usikkerheden ved at slippe styret, hvor det på nogen cykler også betyder at de ikke kan bremse samtidig, skader eller andre mén som ikke gør det muligt at svinge armen op i den korrekte stilling, eller også bare ren dovenskab.

Kunne der opfindes en sensor med integreret lys signaler, som der selv kunne opfange hvornår cyklisten svingede og tændte det korrekt blink lys. Men også opfange hvornår cyklisten bremses og tænder bremselyset, eller hvornår de bare cykler lige ud af cykelstien og bare skal have normalt baglys tændt.



3 Projekt beskrivelse

Til dette ETTML projekt, vil der blive undersøgt muligheden for at kunne designe et cykellys til at hænge bagpå cyklen, og tænde det nødvendige lys i de forskellige situationer.

Projektet vil bygge på en klassifikations Machine Learning model som skal kunne klassificere mellem 4 klasser. Left, Right, Brake og Straight.

Note: Under implementation er der blevet lavet en stavfejl (Break vs Brake) som går igennem alt data indsamling, model dannelse, model træning samt test og verifikation.

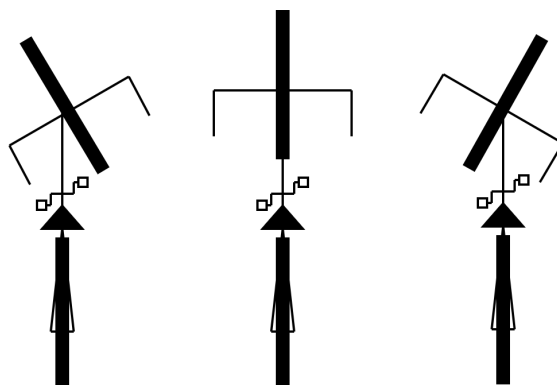
Machine learning modellen skal vælges ud fra test med dannelse og træning af forskellige modeller, hvad der er muligt at implementere med Emlearn [3] (Framework for implementaion af Machine learning modeller i C kode, konverteret fra en python model) samt hvad der er lært fra undervisningen omkring hvor det er favorabelt at bruge de forskellige modeller til.

Under projektet vil der blive lavet data indsamling, analyse samt validering af data. Forholde sig kritisk til datatyper samt variationen af data, for at give Machine Learning modellen den bedst mulige forudsætning for at kunne vælge de korrekte klasser.

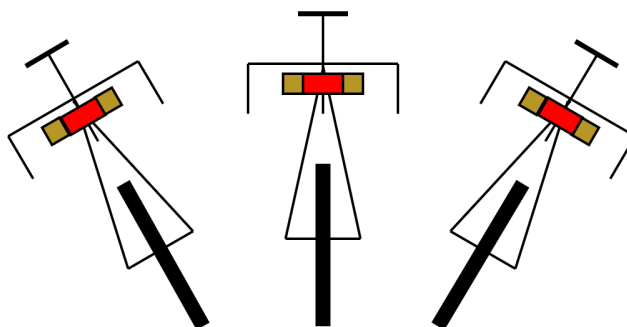
Den fulde github kan findes her [?], med alt kode, opsamlet data og andet anvendt til projektet.

4 Krav analyse

En cykel er en kompleks enhed som der er bygget op omkring forskellige geometrier. Når man cykler lige ud, er vægten fordelt midt på cyklen over hjulene. Men når man drejer på en cykel, så drejer man styret men også vægt fordelingen på cyklen. Fra at centrum af massen er over cyklen, rykkes den nu ud til siden for at sikre at cyklisten ikke vælter over cyklen fordi momentummet er i den fremad gående retning. Dette er skitseret på Figur 1 og Figur 2 for hvordan cyklen tilter med sving henholdsvis til vestre og højre.



Figur 1: Skitse af cykel oppe fra når der drejes



Figur 2: Skitse af cykel bagfra med lys monteret.

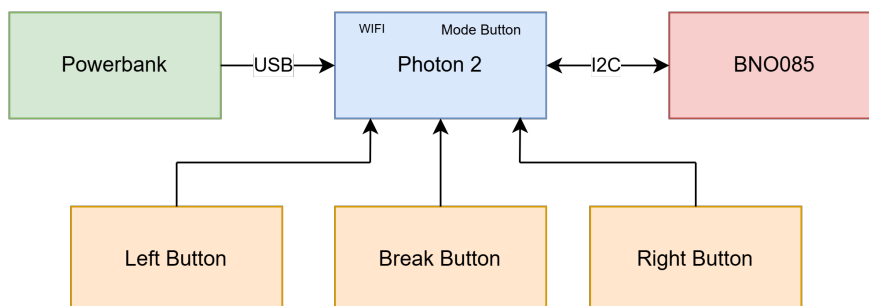
Som det kan ses på skitserne, så bliver det en vinkel ændring, både styrets ændring men også cyklens vinkel som der skal opfanges af en sensor. Til dette ville et gyroskop være god, da den fanger ændringen af vinklen over tid i radianer eller grader pr sekund.

For at kunne fange bremsningen af cyklene, eller en deacceleration, skal vi bruge et accelerometer, som der kan fange dette.

Der findes en sensor som hedder BNO085, som der både har acceleromter, gyroskop men også magnetometer indbygget. Sensoren bygger op omkring en Cortex M0+ som der opsamler data, formaterer og gør det muligt for brugeren at hente det i forskellige formater. Dette vil jeg komme mere ind på senere, da data opsamlingen er blevet testet med flere forskellige datatyper.

5 System design

Efter at have valgt en passende sensor, som der kan opsamle de forskellige data som vi skal bruge, er der blevet lavet et blokdiagram for at vise snitfladen mellem sensoren og Photon2, men også forskellige extra ting for at kunne lave simpel og effektiv data indsamling. Blokdiagrammet er vist herunder på Figur 3:



Figur 3: Blok diagram over systemet

Particle Photon 2, er en smart lille microcontroller som der har wifi inbygget samt flere forskellige snitflader og I/O ben for nemt at kunne integre med den. Der bliver brugt I2C som snitflade til BNO085 for nemt at kunne hente data, samt flere forskellige knapper til at kunne label de opsamlede data, og tilhørende debounce kredsløb for at undgå kontakt præl. Til den færdige implementaion er det kun powerbank, photon 2 og BNO085 som der sat til systemet. Den fysiske implementation kan ses på Figur 4



Figur 4: Billede af den fysiske implementation af systemet

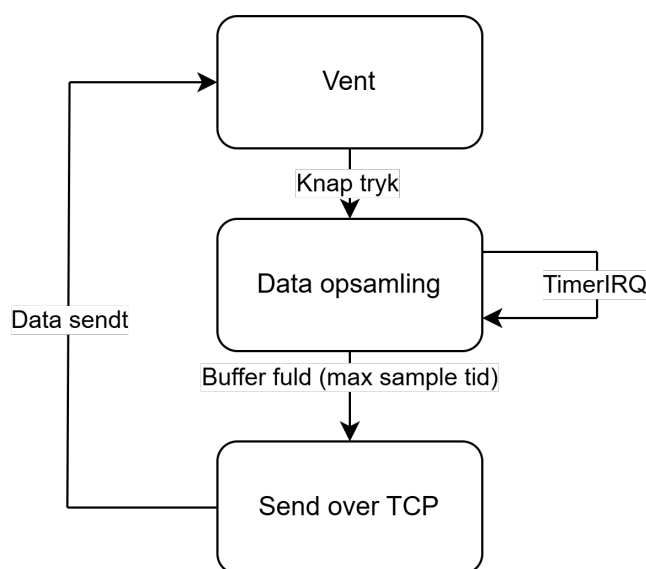
6 Implementation

6.1 Data indsamling

Til at lave data indsamling og kunne danne et relevant data sæt, er der taget udgangspunkt i øvelsen accelerometer data over TCP [14]. Kildekoden er blevet ændret for at kunne bruges til den nye sensor, samt gøre det nemmere at navngive de forskellige klasser. Data indsamlingen bygger på Photon 2 der indsamler data med et fast interval, sammensætning af et tidsstempel og dataen fra sensoren i henholdsvis X-, Y-, og Z-akser.

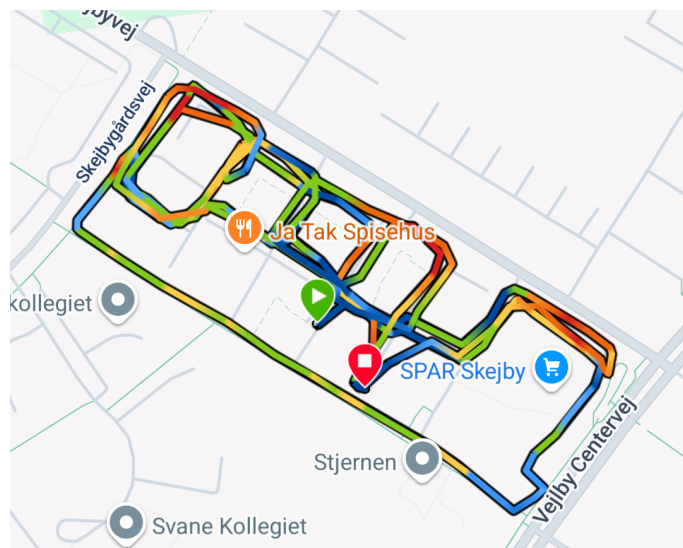
Til at kunne interface med den nye sensor, blev der brugt et bibliotek til at gøre data indsamlingen nemmere. Biblioteket [7] er lavet af SparkFun, som også sælger et modul af den valgte sensor.

Koden bygger på en state machine, som der startes efter et knap tryk. Dette starter dataopsamlingen og fortætter indtil bufferen er fuld også overføres dataene, over TCP, til serveren der konvertere dem til CSV. Så stopper dataindsamlingen og sættes tilbage i vente position til næste knap tryk. Dette er afbildet på Figur 5:



Figur 5: State machine for data indsamling

Den samlede kode kan ses i BNO085_to_TCP_server.cpp på github [15]



Figur 6: Den cyklede rute for at opsamle data

TCP serveren bliver startet op med at give et navn på output filen, hvor den så selv numerer filerne efter antal data serveren har modtaget. For at gøre det nemmere ikke at kun skulle dreje til højre hele tiden, og skifte klasse senere, er knapperne blevet implementeret til at ligge navn ind i headeren af csv output filen, til at senere kunne ændre i fil navngivning som passer med navnet på klassen den opsamlede data kommer fra. Et eksempel på headeren kan ses på Listing 1:

```

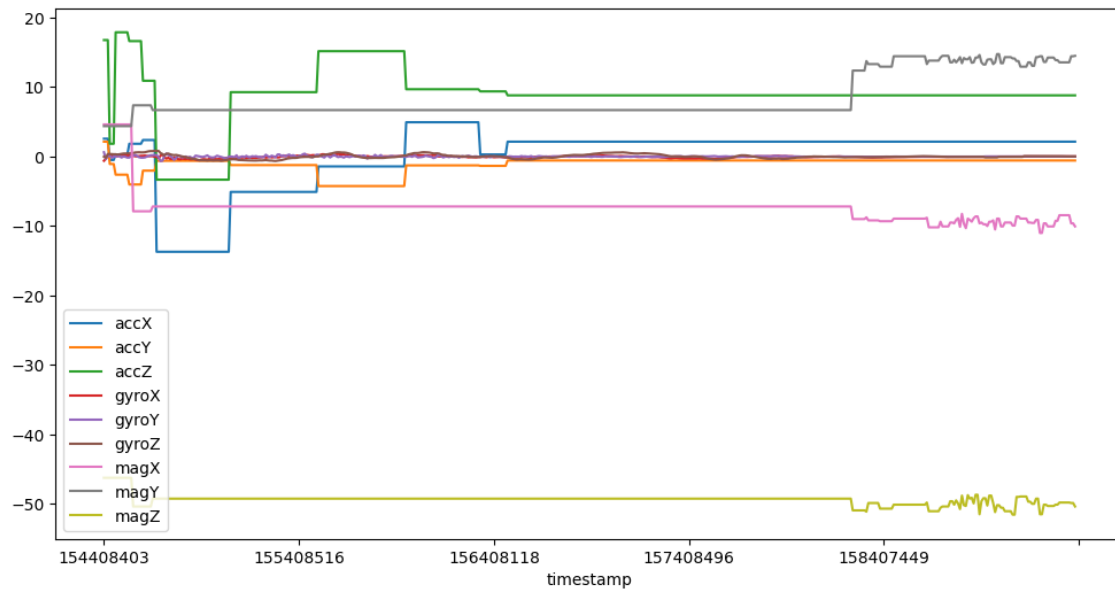
1  timestamp,accX,accY,accZ,gyroX,gyroY,gyroZ
2  Right
3  7,2.45,2.53,8.85,-0.01,0.00,0.00
4  19,2.45,2.53,8.85,0.00,0.00,-0.00
5  21,2.45,0.34,10.57,0.00,0.00,-0.00

```

Listing 1: Eksempel på header i .csv output fil.

BNO085 indeholder også magnetometer som indsamler en retning iforhold til jordens magnetfelt. For at have flest mulige datapunkter og kunne regne features ud senere, blev det valgt også at prøve at inkludere denne sensor i data indsamlingen.

Det først indsamlede data kan ses på Figur 7:



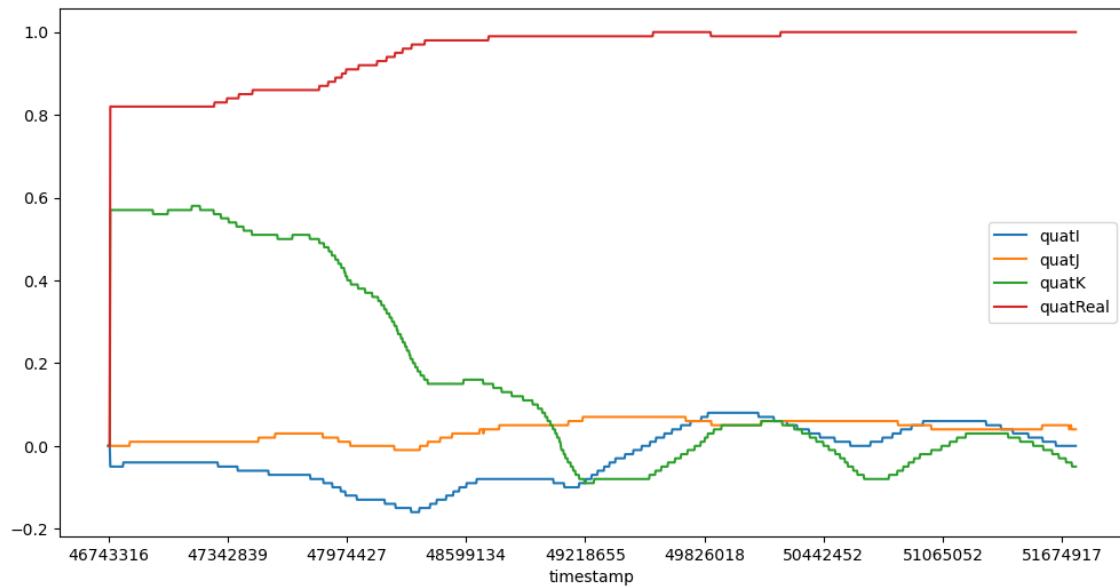
Figur 7: Indsamlet data fra Accelerometer, Gyroskop og Magnetometer

Det kunne hurtigt konkluderes at sensoren blev overloaded og sensoren ikke havde nået at opdatere data, med det interval vi prøvede at polle med. Efterfølgende blev det fundet i databladet for BNO085 [4] Section 6.9 omkring report rates, at den maximale datarate Tabel 1 for accelerometer, gyroskoper og magnetometeret lagde på henholdsvis 500, 400 og 100 Hz. Data indsamlingen var forsøgt med 100Hz for at tilgode se den langsomste sensor, men dette var ikke nok når der skal bruges andre sensorer også. Derfor skulle sample raten sættes så langt ned for at alle sensorerne kunne nå at blive samlet, at dette ikke var en god mulighed for projektet.

Composite Sensor	Maximum Data rates (Hz)
Gyro rotation Vector	1000
Rotation Vector	400
Gaming Rotation Vector	400
Geomagnetic Rotation Vector	90
Gravity	400
Linear Acceleration	400
Accelerometer	500
Gyroscope	400
Magnetometer	100

Tabel 1: Figur 6-16 fra BNO085 Databladet

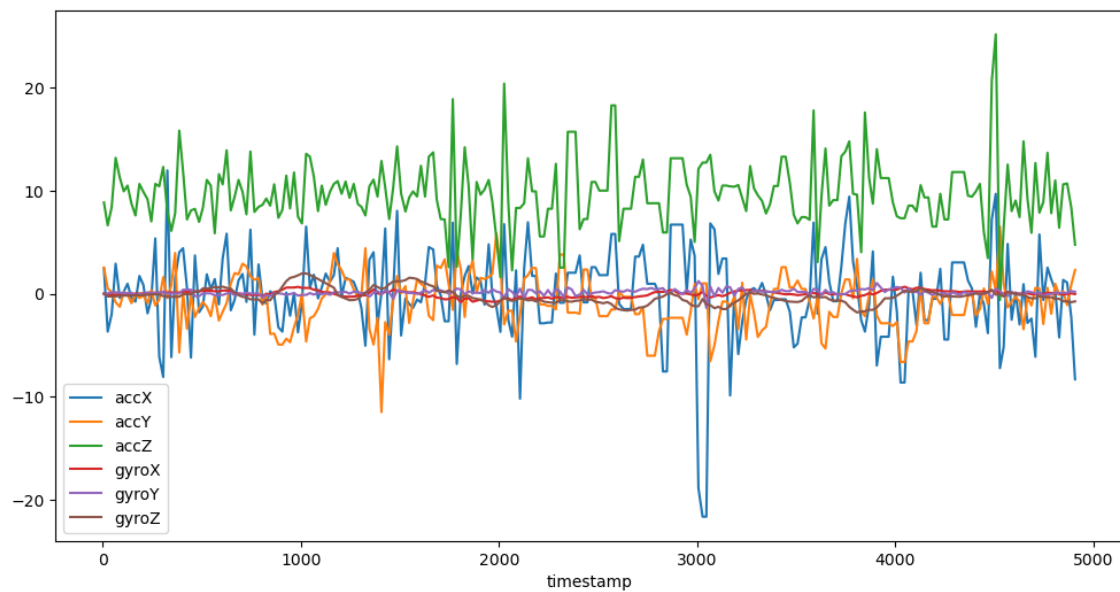
Efterfølgende blev der prøvet med Rotations Vektoren (figure 2 i SH-2 reference manual[4] viser sammensætningen af sensorer og data output) for at kunne komme op på en højere sample rate, men så også fravælge magnetometeret som alligevel ikke var nødvendig for projektet. Den indsamlede data fra Rotations Vektoren kan ses på Figur 8, med en samplerate på 333 Hz:



Figur 8: Indsamlet data fra Rotation Vector (Accelerometer og Gyroskop samlet)

Rotations vektoren har akserne quatI, quatJ, quatK og quatreal, som der bliver brugt for de forskellige bevægelse retninger. Quat bliver brugt om en bevægelse som er ligeglad med rækkefølge af den sammensatte bevægelse, hvor imod X-, Y- og Z-akse bevægelse er nødvendig i den rigtige række følge. Forklaringen stammer fra [5] & [6]

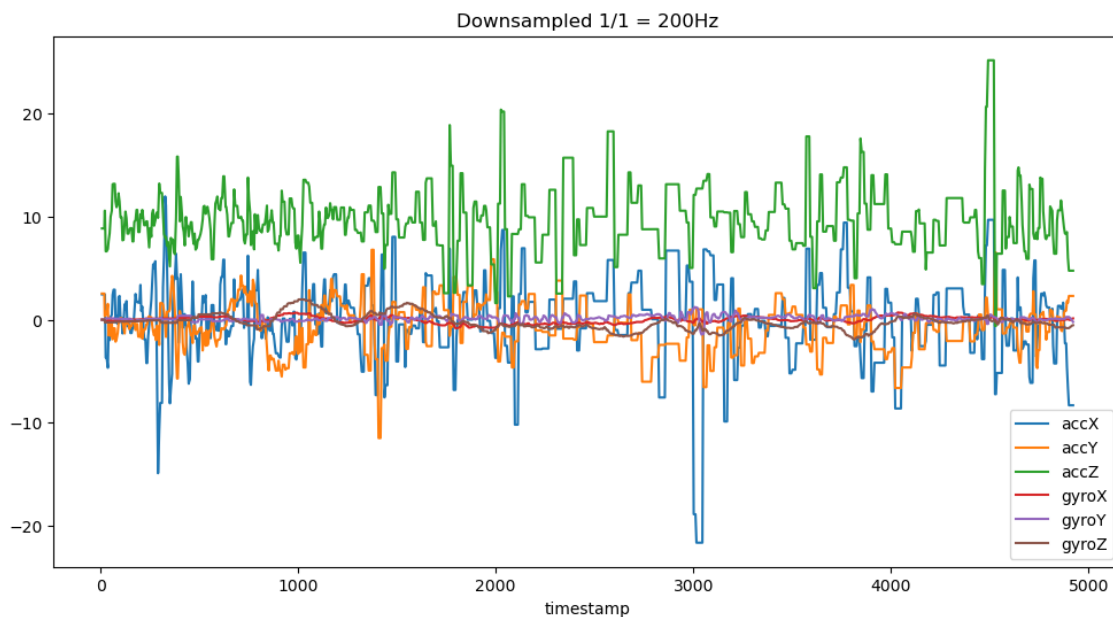
Det opsamlede data er blevet testet igennem Edge Impulse [8] for at tjekke for brugbarheden af dataene. Det var ikke særlig vellykket med quat datane, og derfor blev det valgt at gå tilbage til hver enkelt sensor (Accelerometer og Gyroskop) med sample rate på 200Hz for at sikre en stabil og korrekt data.



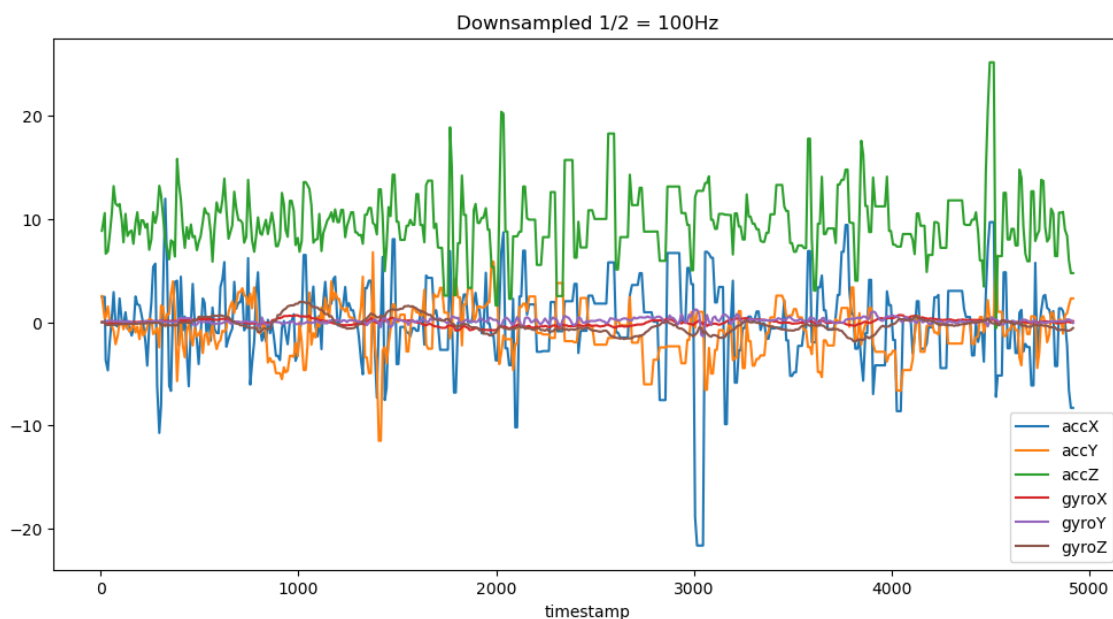
Figur 9: Indsamlet data fra Accelerometer og Gyroskop

6.2 Data nedsampling

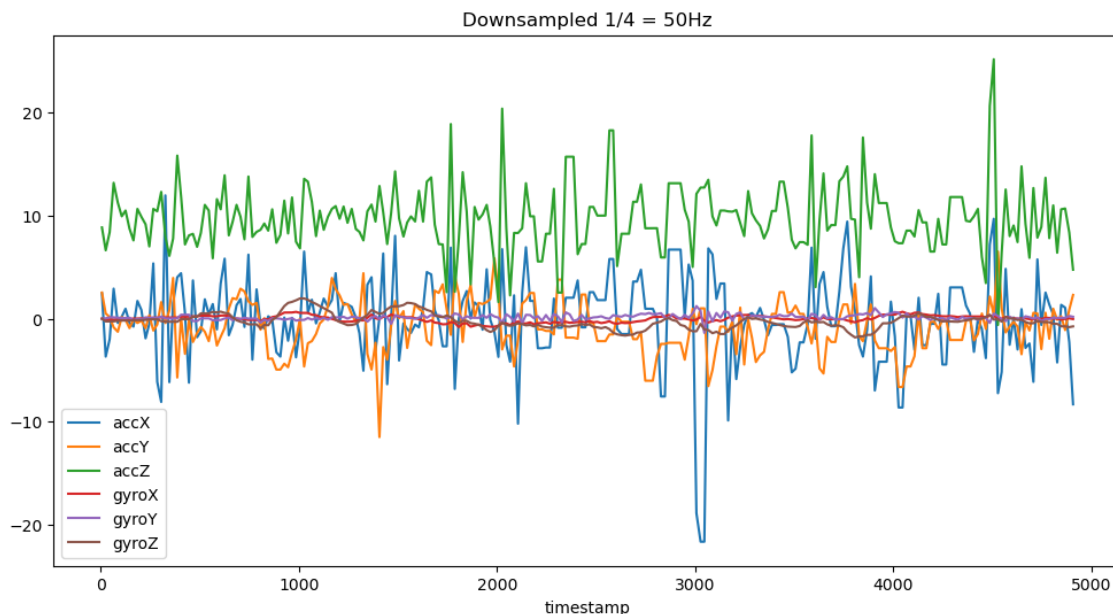
I undervisningen er der blevet vist forskelle i hvor hurtigt man sampler og hvilken værdi det kan give, at oversample og derefter nedsample, så man får mere data at kunne træne med. Dette er blevet testet på det indsamlede data for at se efter om dette også var et trick der kunne bruges i dette projekt. Samtidig med at data filerne blev omdøbt fra csv headeren, blev en af filerne nedsamplet til henholdsvis 200 Hz, 100 Hz og 50 Hz. På de efterfølgende plots kan der ses de nedsamplede plots Figur 10, Figur 11 og Figur 12:



Figur 10: Rå data



Figur 11: Data nedsamplet til 100 Hz



Figur 12: Data nedsamlet til 50 Hz

Der kunne ikke vurderes nogen nævenyttig forskel på de 3 plots, til alt dataene er blevet nedsamlet til 50 Hz for at øge data mængden. Når dataen skal splittes op i træning og test er der blevet taget højde for at hele original filer (4 nedsamlet datafiler) skal i enten træning eller test for ikke at træne og test på samme data.

6.3 Edge Impulse

På opfordring af underviseren Morten, er Edge Impulse blevet brugt til at validere data og se hvordan deres modeller opfører sig på dataene. Edge Impulse er et simpelt og meget nemt tilgængeligt værktøj som der hurtigt og enkelt kan lave en machine learning model til dig.

Jeg oprettede et impulse, importede data, opdelt i træning og test data, indtastede det var samlet med 50 Hz (efter nedsamlingen). Der blev lavet feature extraction og score af features samt trænet et neural netværk. Dette netværk blev der så lavet en test af hvor der kom konfusion matrix ud og score på hvor god hele modellen var. Der kommer også plot over alle datapunkterne med hvilke den klassificerede korrekt og hvilke den ikke gjorde.

De mest relevante features som Edge Impulse havde brugt var:

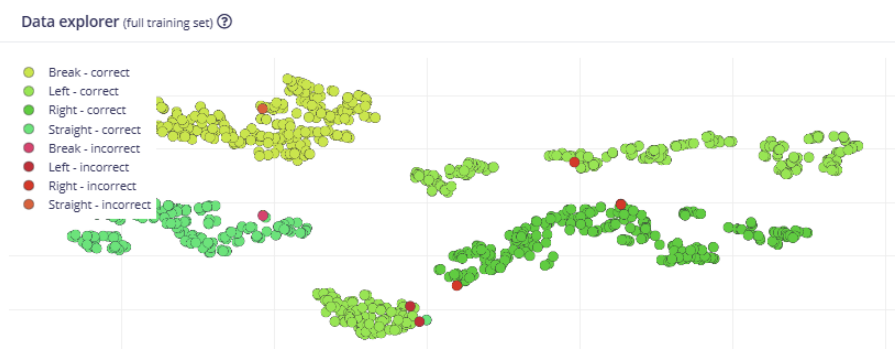
1. gyroZ Average
2. gyroX RMS
3. accY Average
4. gyroY RMS
5. gyroZ Minimum

Tabel 2: De 5 vigtigste features som Edge Impulse har brugt

Og plots over output data fra test af det neurale netværk:



Figur 13: Konfusion matrix for den trænte Edge Impulse model



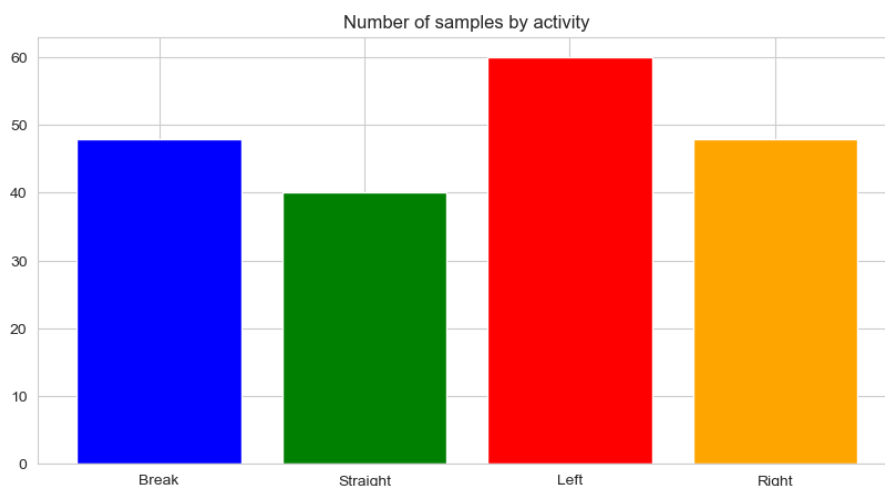
Figur 14: Plot over klassificerede datapunkter

Edge Impulse viser en flot klassifikations model med en nøjagtighed på 99%. Det er meget pænt og man kan se hvor de forskellige datapunkter ligger spreadt ved hjælp af feature extraction, så vi visuelt næsten også kan klasificere dem.

Dette skal nu prøves at efterlignes med egen lavet model og feature extraction ved hjælp af python.

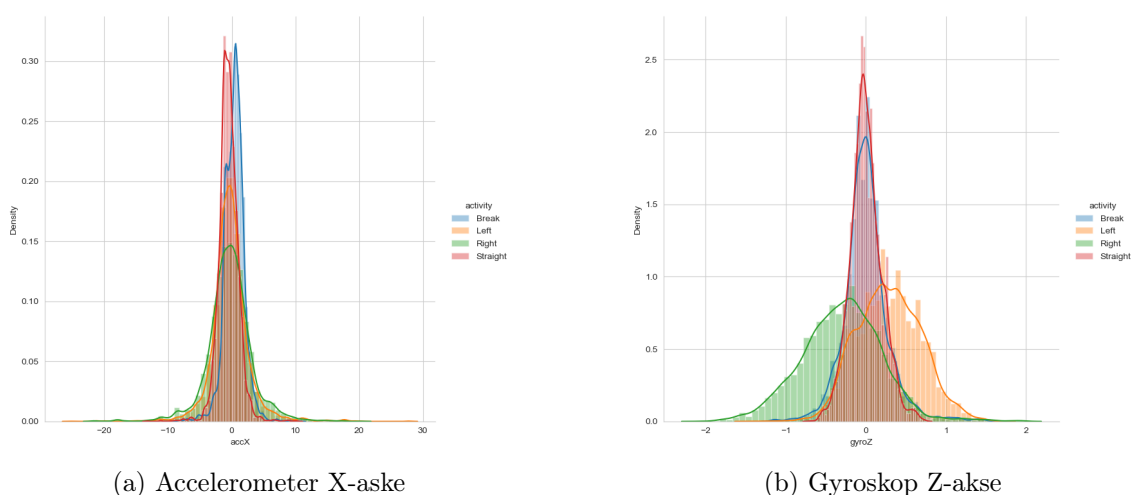
6.4 Data behandling

Alt det nedsamlede data er blevet importeret i python, med aktivitet for hvad der sker under de samples (straight, left, right eller break), og index for hvor mange samples der er af hver type. Dette er blevet vist et søjlediagram (Figur 15) som viser en nogenlunde ligelig fordeling af forskellige aktiviteter.



Figur 15: Søjlediagram over antallet af de forskellige aktiviteter

For at kunne visualiser forskelle imellem de forskellige aktiviteter, kan man lave et histogram over datapunkterne. Disse er lavet på alle 6 data typer, men der er 2 der står ud med forskelle. Det ses på Figur 16:



Figur 16: Histogrammer over data punkter

Der kan det ses på accelerometeret i X-aksen (bevægelses retningen) at der er forskel mellem straight og break som passer meget overens med analysen der blev lavet under Krav analyse. Det samme kan ses på gyroskopets Z-akse som skifter alt efter om cyklen vipper/drejer til left eller right.

Efter dette bliver dataene delt op i train og test data, for at kunne gemme ca 20% til test af de forskellige modeller som vi danner. Det er 20% af hver aktivitet, så det er ligeligt fordelt procentvis.

Og efterfølgende bliver der lavet vinduer og features extration på alle train dataene. Vinduerne har en størrelse på 100 sampels ($100 * 1/50 \text{ Hz} = 2 \text{ sekunders data}$) med et overlap på 50 sampels. Dette giver nok data til at få en god feature udregning, men



med overlaop på 1 sekund blive responsen stadig hurtig. Det er taget udgangspunkt i alle tænkelige features som vi har brugt i en øvelse, som fulgte Mediums guide [9]. Der bliver i alt 118 forskellige features som der er regnet på. Nogen er akse og sensor specifikke og andre er en sammensætning af alle akserne under sensoren.

6.5 ML model analyse samt træning

For at kunne vælge mellem de forskellige modeller, skal vi have et overblik over dem. Der skal dannes en klassifikations model, så fra undervisningens slides [10] har vi flere forskellige at vælge imellem. Men for at kunne implementere dem, har jeg valgt at gå med emlearn og de modeller det understøtter. Derfor er det valgt at stå imellem KNN (k-Nearest Neighbors) og Random forest modellerne. Vi nu prøve at lave en model af dem begge og se hvordan de bliver.

Dette bliver gjort ved hjælp af scikit learn [11]. Scikit learn er et python add-on som både består af forskellige Machine learning modeller men også scalere og alt andet anvendeligt til machine learning.

Starter med at lave en KNN model, ved at bruge standart scaler (fordi vi ikke har scaleret noget af vores data endnu, og få bedst udnyttelse af data range), og derefter anvende train features til at træne modellen på. Modellen bliver trænet og efterfølgende bliver den testet med de 20% data om der var gemt til test.

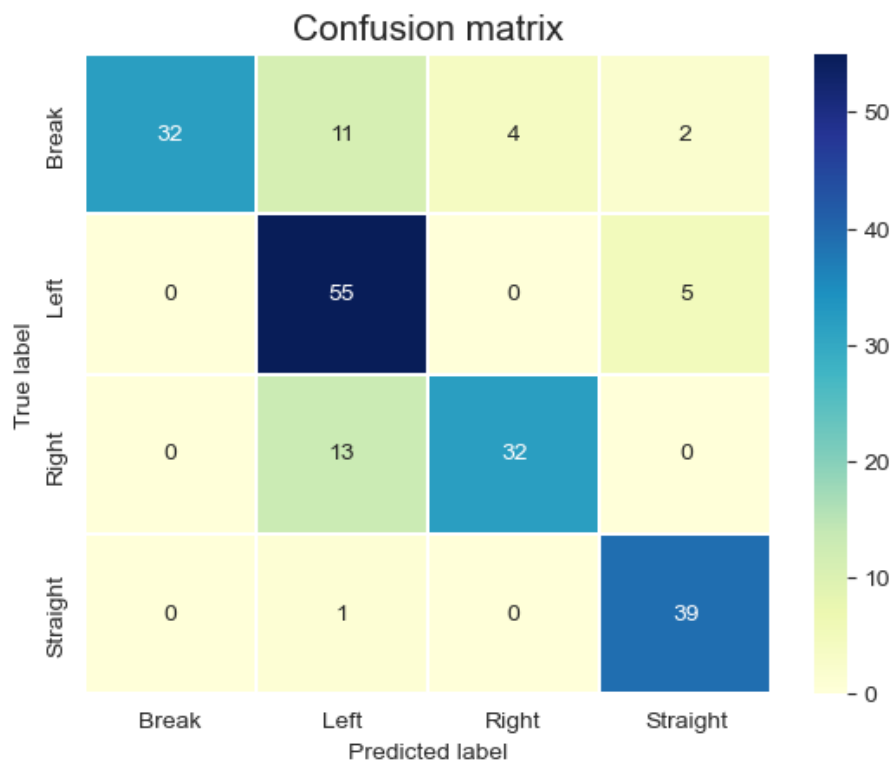
```
1 # standardization
2 scaler = StandardScaler()
3 scaler.fit(X_train)
4 X_train_data_lr = scaler.transform(X_train)
5 X_test_data_lr = scaler.transform(X_test)
6 # k-Nearest Neighbors model
7 knn = KNeighborsClassifier(n_neighbors=11)
8 knn.fit(X_train_data_lr, y_train)
9 y_pred = knn.predict(X_test_data_lr)
10 print("Accuracy:", accuracy_score(y_test, y_pred))
```

Listing 2: knn model, trænet samt lavet test score

Ud fra dette får vi en score på $0.8144329896907216 = 81\%$ success rate. For at visuelt se dette kan vi også lave en confusion matrix:

```
1 labels = ['Break', 'Left', 'Right', 'Straight']
2 confusion_Matrix = confusion_matrix(y_test, y_pred)
3 sns.heatmap(confusion_Matrix, xticklabels=labels, yticklabels=labels, annot
   =True, linewidths = 0.1, fmt="d", cmap = 'YlGnBu')
4 plt.title("Confusion matrix", fontsize = 15)
5 plt.ylabel('True label')
6 plt.xlabel('Predicted label')
7 plt.show()
```

Listing 3: knn model, confusion matrix



Figur 17: Confusion matrix KNN

En confusion matrix er et godt værktøj til at kigge på hvilke klasser der kan være problemer med og hvilke den bare genkender godt. Største delen af labelse skal ligge på diagonalen for at have gættet korrekt.

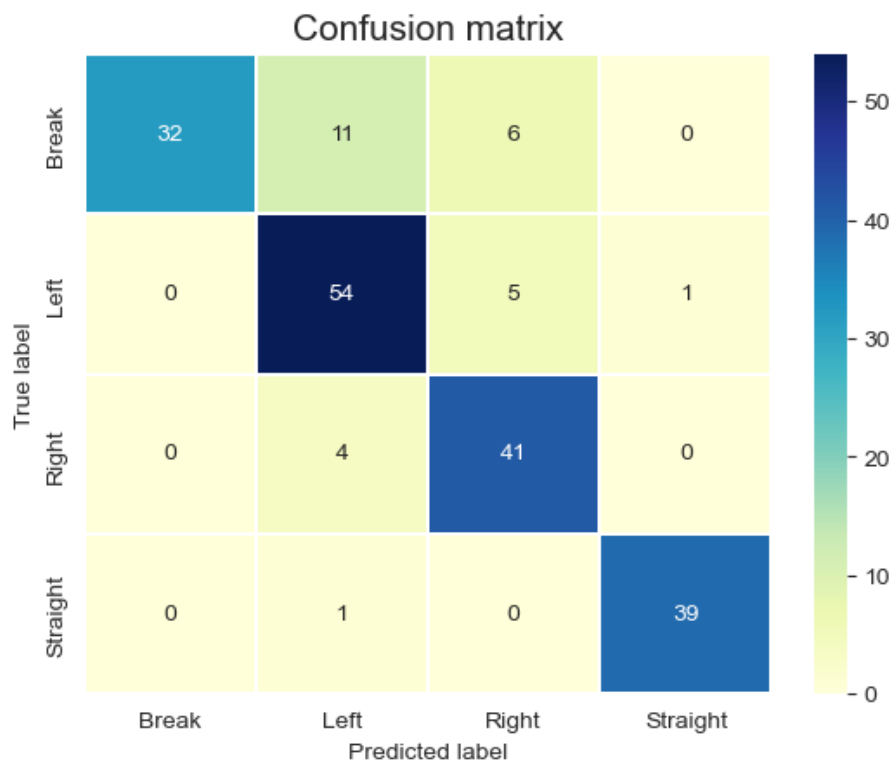
Det samme kan gentages sig med en random forest model. Det er samme scalerede data som der bliver brugt til random forest.

```

1 # Train the Random Forest model
2 rf = RandomForestClassifier(random_state=42, n_estimators=100)
3 rf.fit(X_train_data_lr, y_train)
4
5 y_pred = rf.predict(X_test_data_lr)
6
7 # Evaluate the model
8 accuracy_before = rf.score(X_test_data_lr, y_test)
9 print(f'Accuracy before feature selection: {accuracy_before:.2f}')
```

Listing 4: Random forest model, trænet samt lavet test score

Ud fra dette får vi en score på $0.8556701030927835 = 86\%$ success rate. Dette er bedre end vores KNN model og dette kan også ses på confusion matrix for random forest Figur 18, da flere labels ligger på diagonalen. Det er især omkring forskellen mellem left og right, hvor random forest klarer sig bedre.



Figur 18: Confusion matrix Random forest

Nu er det jo 118 features som vi regner på, som er en del for hvad der skal bruges til at kunne genkende de forskellige klasser. Derfor findes der en feature importance i vores random forest model. Den kan udvælge og se hvilke features som der vægter mest i modellen:

```

1  # Extract feature importances
2  importances = rf.feature_importances_
3  feature_names = X_train.columns
4  feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
5
6  # Rank features by importance
7  feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
8  print(feature_importance_df[:10])
9
10 # Select top N features (example selecting top 10 features)
11 top_features = feature_importance_df['Feature'][:10].values
12 X_train_selected = X_train[top_features]
13 X_test_selected = X_test[top_features]
14
15 print(top_features)

```

Listing 5: Random forest model, feature importance

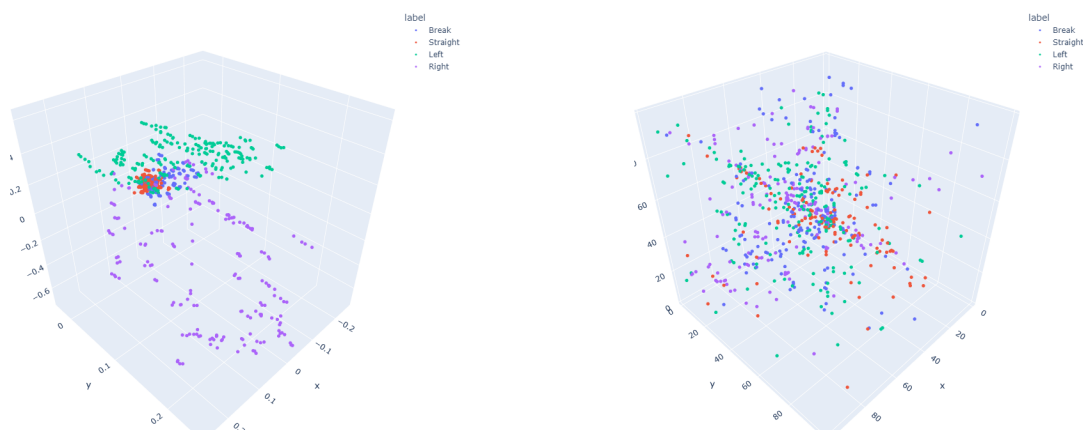
Ud fra dette er der kommet 6 features som er vigtigst for random forest modellen:

feature number	Feature	Importance
5	zg_mean	0.0621
99	sma_gyro	0.0486
27	xg_max	0.0429
0	xa_mean	0.0419
1	ya_mean	0.0401
50	za_IQR	0.0363

Tabel 3: De højeste vægtede features for random forrest modellen

Hvis i sammenligner Tabel 2 og Tabel 3, kan vi se de begge har gyro Z gennemsnit som den vigtigste feature, men også acceleration Y gennemsnit i blandt top 5.

Featuresne fra Tabel 3 er dem som vi senere kan bruge at implementere på vores Phonton 2 til at bruge modellen. For at visuelt kunne se de valgte features, hvordan deres data er og hvordan de bliver brugt i modellerne kan vi laver scatter plots af hver enkelt feature men også i forhold til modellen:



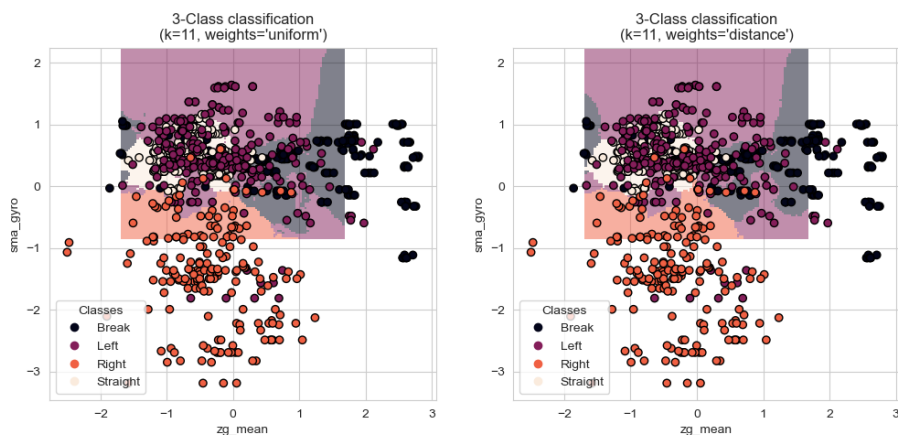
(a) Gyro mean for X-, Y-, og Z-akse

(b) Gyro arg diff for X-, Y-, og Z-akse

Figur 19: Scatter plots af features

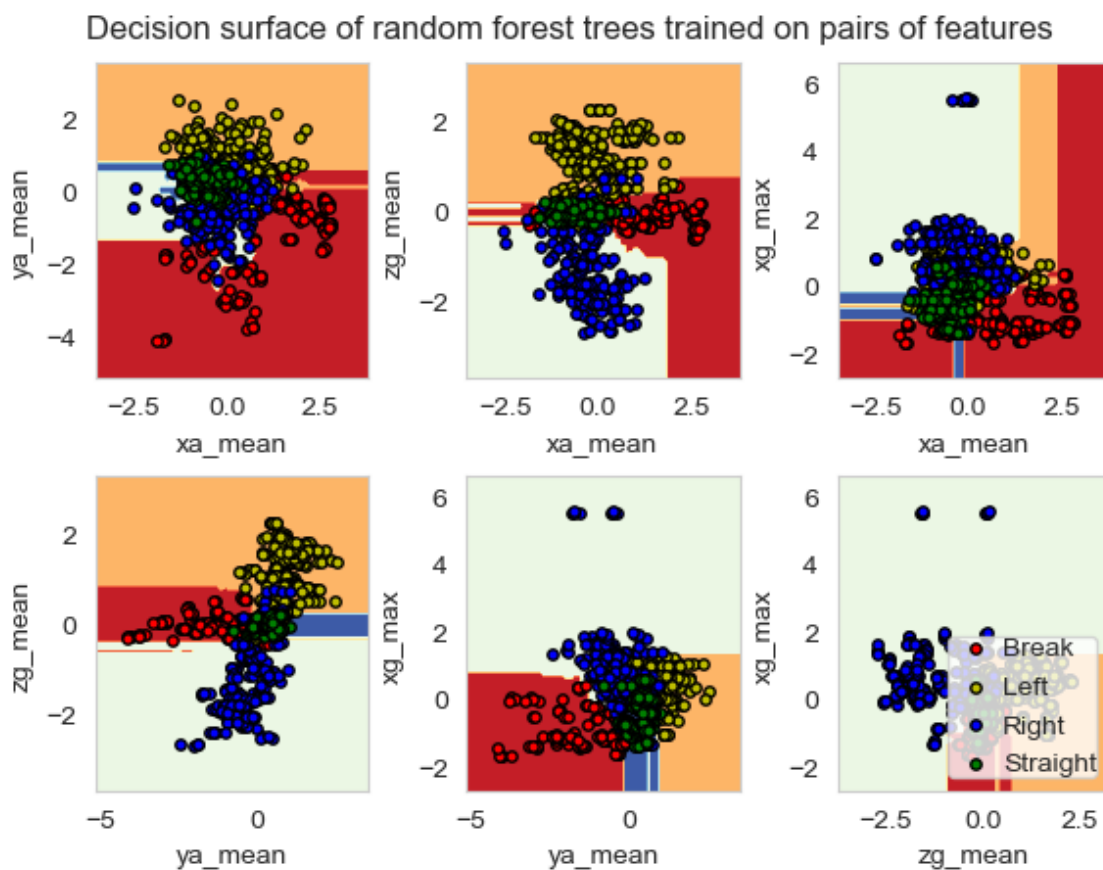
På Figur 19 kan vi se et 3D plot af alle feature punkterne for de valgte vinduer, for henholdsvis gennemsnit af gyroskopet, og forskellen i index mellem max og min værdi for tids domænet. Denne feature havde importance på 0 og derfor ikke nogen random forest modellen har gavn af. Det ses tydeligt hvor nemt det er for gennemsnittet at tegne ringe omkring de forskellige labels, imens arg diff er spredt ud over det hele.

For KNN modellen er der lavet et scatter plot omkring 2 features, men også forskellen i at bruge uniform eller distance som vægtning. Ved uniform ser modellen på hvor mange punkter af hver klasse der er inden for en given radius, og klassifisere ud fra det. Imens på distance udregnes den mindste længde til et punkt i hver klasse og klassifiseres ud fra det.

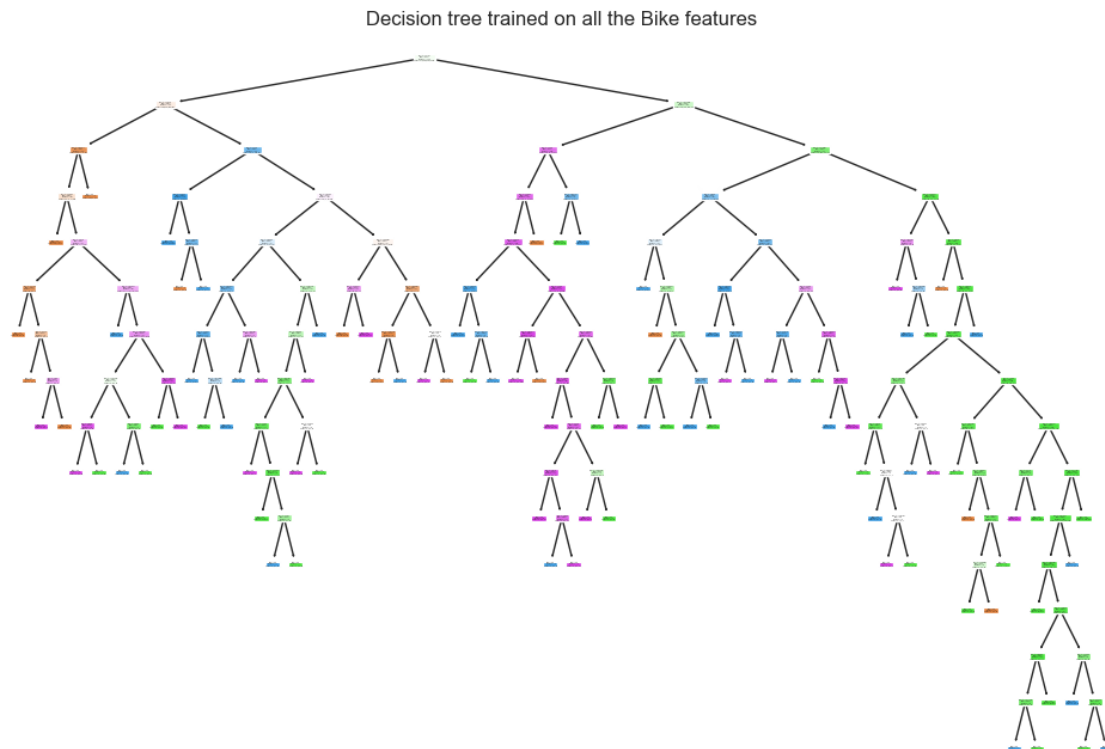


Figur 20: Scatter plot mellem 2 features, og forskellen på distance eller uniform fordeling

I scatter plottet for random forest, tegnes der nogen felter for, hvilke værdier de forskellige klasser er indenfor. Random forest, kigger altså på om feature værdien er højere eller lavere end et sætpunkt også vælger ud fra dette, flere gange med forskellige features. Dette kan også ses på Figur 22, som viser alle grenene som modellen skal vælge ned af.



Figur 21: Scatter plot mellem 4 features



Figur 22: Random forest plot over alle grenene i et af træerne

Det samme ses for dem, at med de korrekte features, kan man selv også visuelt se forskellen mellem de forskellige datapunkter og klassificere dem. Nogen klasserne ligger ovenpå hinanden, men det er derfor det er mellem alle features og ikke som det er, afbilledet mellem 2 at klassifikationen sker.

7 Test & Verification

7.1 Emlearn

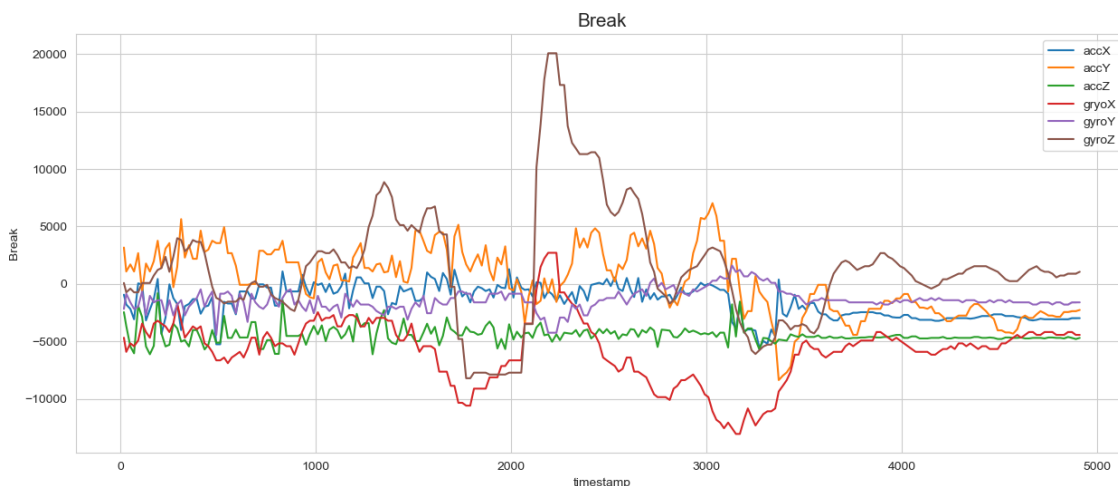
Som tidligere nævnt er det Emlearn jeg gerne vil anvende til at køre Machine learning modellen på Photon 2. For at bruge Emlearn er det et krav at alle dataene er signed int16, og derfor er dette den scaler som vi skal bruge.

Til dette har Scikit learn også en MinMaxscaler [12] som der kan bruges. Der kan vi give interval for hvordan vi gerne vil have dataene scaleret. Scaleren kan tage rå data ind og finde max og min for alle de forskellige data typer. Outputtet kan ses her:

	accX	accY	accZ	gyroX	gyroY	gyroZ
Max	27.81	11.26	40.5	1.52	1.82	1.96
Min	-25.21	-14.18	-14.25	-1.14	-1.65	-2.07

Tabel 4: Max og min værdier for alle data typer

For at være sikker på at sikker på vi ikke klipper i noget af dataene, og får forkerte værdier ud, er der lagt en extra buffer ind, så max og min bliver sat til henholdsvis ± 50 for accelerometer data og ± 5 for gyroskop data. Dette gør det også nemmere for os senere når det skal implementeres på Photon 2. Dette gør faktisk også at vores vores data er nemmere at se visuelt, da værdierne for acclerometeret og gyroskopet ikke har være ens i range som ses på Figur 23



Figur 23: Plot af scaleret break opsamlings data

Efter at have scaleret alle dataene kan vi køre igennem feature extraction igen og træne modellerne, og se om der er en forskel på nøjagtigheden for modellerne.

	KNN	Random forest
Før	81%	86%
Efter	82%	78%

Tabel 5: Nøjagtighed før og efter 16bit skalering



Vi kan faktisk se at KNN modellen bliver lidt bedre, imens random forest bliver dårligere af skaleringen.

Nu kan vi så prøve, ved hjælp af Emlearn, at konvertere modellerne over til C-kode og se hvordan de så performer, ved efterfølgende sammenligne dem med python modellen ved hjælp af en f1-score:

```
1 import emlearn
2
3 cmodel_rf = emlearn.convert(rf, method='loadable')
4
5 cmodel_rf.save(file='./random_forest_model.h', name='random_forest_model')
6
7 from sklearn.metrics import f1_score
8
9 pred_py_rf = rf.predict(X_test)
10 pred_c_rf = cmodel_rf.predict(X_test)
11
12 activitys = ['Break', 'Left', 'Right', 'Straight']
13
14 temp = []
15
16 for i in pred_c_rf:
17     temp.append(activitys[i])
18
19 pred_c_rf = temp
20
21 val_score_rf = f1_score(y_test, pred_py_rf, average='weighted')
22 c_val_score_rf = f1_score(y_test, pred_c_rf, average='weighted')
```

Listing 6: Model convertering til C-kode samt sammenligning

Python	C
78%	79%

Tabel 6: F1-score mellem python og C modellen

Det er en meget sammelignlig score som siger at de 2 random forest modeller performere ens. Dette kunne desværre ikke efterlignes for KNN modellen da VS code crashede hver gang jeg prøvede at konvertere den model. Derfor bliver resten af implementationen og test kun udført med random forest modellen.

Emlearn kommer ud med et tal alt efter klassen som der efterfølgenden bliver konverteret til et label. Det er vigtigt at man ikke kommer til at bytte rundt på labels ellers får man en forkert score, som jeg da jeg havde byttet rundt på alle labels og fik en score på under 50%.

7.2 C kode

Efter at have omdannet modellen til en model i C-kode, kan vi nu implementere den på python 2. Alt det skalering og feature extraction skal nu implementeres om C-kode som det var skrevet i python.



Udfra scikit learns MaxMinScaler [12], bruges deres udregning til skalering i C:

```
1 std_accX = (data1->accX - accMin) / (accMax - accMin);  
2 data[0][i] = (int16_t)(std_accX * (32767 - -32768) + -32768);
```

Listing 7: Skalering af data på photon 2

Efterfølgende skal der så laves feature extraction på de scaledede data. Dette er nemt for nogen af featuresne som f.eks. `xa_mean`, `ya_mean`, `zg_mean` og `xg_max` med simpel C-kode:

```
1 for(int i = 0; i < data_length*2; i++){  
2     x_mean += data[0][i]; // xa_mean  
3     y_mean += data[1][i]; // ya_mean  
4     z_mean += data[5][i]; // zg_mean  
5 }  
6  
7 features[0] = (int16_t)(x_mean / (data_length*2)); //xa_mean  
8 features[1] = (int16_t)(y_mean / (data_length*2)); //ya_mean  
9 features[2] = (int16_t)(z_mean / (data_length*2)); //zg_mean  
10  
11 for(int i = 0; i < data_length*2; i++){  
12     if (data[3][i] > xg_max){  
13         xg_max = data[3][i];  
14     }  
15 }  
16  
17 features[3] = xg_max; //xg_max
```

Listing 8: Feature extraction i C

Hvor det er lidt sværere med `za_IRQ` og `sma_gyro` da der skal bruges flere matematiske funktioner. For udregningen af `za_IRQ` er følgende guide blevet brugt [1]. Og for udregning af `sma_gyro` er forklaringen [2] samt `math.h` blevet brugt til udregningen. Desværre har det ikke været muligt at gennemføre feature extractionen på de 2 features uden Photon 2 er crashed.

Derfor er test blevet lavet ved hjælp af feature extraction i python men modellen er kørt i C.

Features lagt ind i photon 2 for right datasæt:

```
1 features[0] = -2889;  
2 features[1] = 6908;  
3 features[2] = -4315;  
4 features[3] = 9829;  
5 features[4] = 6914;  
6 features[5] = 30645;
```

Listing 9: Features Right

Og photonen har korrekt klassificeret de features til right:



Prediction time: 2 ms
Prediction: RIGHT
Done

Det samme er nu gentaget med Left, Break og Straight. Her er resultaterne:

Feature Input	Feature Prediction
Right	Right
Left	Right
Break	Right
Straight	Right

Tabel 7: C model predictions on photon 2

Det ser desværre ud til at C modellen kørende på Photon 2 ikke kan klassificere de features som der bliver skrevet direkte ind.

Alt python kode til skalering, model dannelse og konvertering til C model kan ses i [16]



8 Konklusion

Igennem projektet er der blevet opsamlet data, med forskellige sensorer og enheder for at test mulighederne for forskellige data typer. Der er blevet lavet test af kvaliteten af dataene med flere forskellige værktøjer.

Under projektet er det blevet vist plots over data og features som har givet en større forståelse for hvordan modellen arbejder og klassifisere ved hjælp af features. Der er blevet dannet forskellige modeller og set hvordan de forskellige har klaret sig på den opsamlede data. Der er blevet lavet skalering af data for at kunne imødekomme forskellige krav til de værktøjer og frameworks der er blevet brugt til træning og konvertering af machine learning modeller.

Der har været udfordringer med brugen af værktøj f.eks. python, emlearn, photon 2 som desværre ikke er blevet løst inden for tidsrammen for at kunne lave et fuldendt projekt.

Forsent i projektet var det blevet opdaget af de opsamlede data, som er blevet opsamlet over 2 dage, var blevet splittet i 2 mapper og derfor var data mængden mindre til at kunne udregne features og træning af modeller.

Dette blev opdaget da der blev udregnet features til test på photon 2 igennem python. Data filerne var de sidste i rækken, altså en del af data fra dag 2, som der ikke har været en del af de data som der har været brugt til features og træning. En hurtig inklusion af disse data har vist at random forest modellen, har fået forhøjet en f1-score på 94% for både python og C modellen. Det fiksede desværre stadig ikke problemet med udregning på photon 2, men har vist at den opsamlede og anvendte data mængde godt kunne have været øget med stor fordel.



Litteratur

- [1] Compute the Interquartile Range (IQR) in C
Sidst besøgt: 24/05/2025
<https://labex.io/tutorials/c-compute-the-interquartile-range-iqr-in-c-435160>
- [2] Signal magnitude area
Sidst besøgt: 25/05/2025
https://en.wikipedia.org/wiki/Signal_magnitude_area
- [3] Emlearn
Sidst besøgt: 25/05/2025
<https://github.com/emlearn/emlearn>
- [4] Ceva SH-2 Reference Manual
Sidst besøgt: 25/05/2025
<https://github.com/RasmusGramstrup/ETTML-Smart-Bike-Light/blob/main/Datasheets/SH-2-Reference-Manual.pdf>
- [4] Ceva BNO085 Datasheet
Sidst besøgt: 25/05/2025
https://github.com/RasmusGramstrup/ETTML-Smart-Bike-Light/blob/main/Datasheets/BNO080_085-Datasheet.pdf
- [5] Quaternion Wikipadia
Sidst besøgt: 25/05/2025
<https://en.wikipedia.org/wiki/Quaternion>
- [6] Quaternions and 3d rotation, explained interactively
Sidst besøgt: 25/05/2025
<https://www.youtube.com/watch?v=zjMuIxRvygQ>
- [7] SparkFun_BNO080_Arduino_Library (community library)
Sidst besøgt: 25/05/2025
https://docs.particle.io/reference/device-os/libraries/s/SparkFun_BNO080_Arduino_Library
- [8] Edge_impulse
Sidst besøgt: 25/05/2025
<https://edgeimpulse.com/>
- [9] Feature Engineering on Time-Series Data for Human Activity Recognition
Sidst besøgt: 25/05/2025
<https://medium.com/data-science/feature-engineering-on-time-series-data-transforming-s>
- [10] Introduction to Machine Learning
Sidst besøgt: 25/05/2025
https://docs.google.com/presentation/d/1210yG2al31Fa40F1YrRy9Bdlbe0YlE99WzhwiiYx8MU/edit?slide=id.g2e3c1980056_0_164#slide=id.g2e3c1980056_0_164
- [11] Scikit learn
Sidst besøgt: 25/05/2025
<https://scikit-learn.org/stable/>



- [12] Scikit learn MaxMinScaler
Sidst besøgt: 25/05/2025
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>
- [13] Github ETTML-Smart-Bike-Light
Sidst besøgt: 25/05/2025
<https://github.com/RasmusGramstrup/ETTML-Smart-Bike-Light/tree/main>
- [14] TCP løsnings forslag fra Morten
Sidst besøgt: 25/05/2025
https://github.com/RasmusGramstrup/ETTML-Smart-Bike-Light/blob/main/solution_343_tcp_server.zip
- [15] Github ETTML-Smart-Bike-Light BNO_TCP server data collection
Sidst besøgt: 25/05/2025
https://github.com/RasmusGramstrup/ETTML-Smart-Bike-Light/blob/main/src/BN0085_to_TCP_server.cpp
- [16] Skalering, model dannelse og konvertering til C
Sidst besøgt: 26/05/2025
https://github.com/RasmusGramstrup/ETTML-Smart-Bike-Light/blob/main/Models_next.ipynb